

project4

March 31, 2026

1. Import Libraries

```
[53]: import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import confusion_matrix, classification_report, \
    roc_auc_score

from sdv.single_table import CTGANSynthesizer
from sdv.metadata import SingleTableMetadata
```

2. Load Dataset & Check Imbalance

```
[54]: data = pd.read_csv("credit_card_fraud_dataset.csv")

print("Dataset Shape:", data.shape)
print(data["IsFraud"].value_counts())
```

```
Dataset Shape: (100000, 7)
IsFraud
0      99000
1       1000
Name: count, dtype: int64
```

3. Preprocessing

```
[55]: Y = data["IsFraud"]
X = data.drop("IsFraud", axis=1)
le1 = LabelEncoder()
le2 = LabelEncoder()
le3 = LabelEncoder()
X["TransactionDate"] = le1.fit_transform(X["TransactionDate"])
X["TransactionType"] = le2.fit_transform(X["TransactionType"])
X["Location"] = le3.fit_transform(X["Location"])
```

4. Train-Test Split & Baseline Model

```
[56]: X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2,
↳ random_state=42)
print(X_train.dtypes)
model = RandomForestClassifier(random_state=42)
model.fit(X_train, Y_train)
```

```
TransactionID      int64
TransactionDate     int64
Amount             float64
MerchantID         int64
TransactionType     int64
Location           int64
dtype: object
```

```
[56]: RandomForestClassifier(random_state=42)
```

5. Prediction

```
[57]: y_train_pred= model.predict(X_train)
y_test_pred= model.predict(X_test)
cm = confusion_matrix(Y_test,y_test_pred)
print("Confusion Matrix" ,cm)
print(classification_report(Y_test,y_test_pred))
```

```
Confusion Matrix [[19787    0]
 [  213    0]]
```

		precision	recall	f1-score	support
	0	0.99	1.00	0.99	19787
	1	0.00	0.00	0.00	213
	accuracy			0.99	20000
	macro avg	0.49	0.50	0.50	20000
	weighted avg	0.98	0.99	0.98	20000

```
c:\Users\TECH CLUB\AppData\Local\Programs\Python\Python313\Lib\site-
packages\sklearn\metrics\_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\TECH CLUB\AppData\Local\Programs\Python\Python313\Lib\site-
packages\sklearn\metrics\_classification.py:1731: UndefinedMetricWarning:
Precision is ill-defined and being set to 0.0 in labels with no predicted
samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\TECH CLUB\AppData\Local\Programs\Python\Python313\Lib\site-
packages\sklearn\metrics\_classification.py:1731: UndefinedMetricWarning:
```

Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

6. Extract Minority Classes

```
[59]: fraud_data= data[data["IsFraud"]==1]
      print(fraud_data.shape)
```

```
(1000, 7)
```

7. Train CTGAN & Generate Synthetic Data

```
[60]: metadata= SingleTableMetadata()
      metadata.detect_from_dataframe(fraud_data)
      synthesizer= CTGANSynthesizer(metadata)
      synthesizer.fit(fraud_data)
      synthetic_fraud=synthesizer.sample(5000)
      print("Synthetic fraud generated:",synthetic_fraud.shape)
```

```
c:\Users\TECH CLUB\AppData\Local\Programs\Python\Python313\Lib\site-
packages\sdv\single_table\base.py:178: FutureWarning: The 'SingleTableMetadata'
is deprecated. Please use the new 'Metadata' class for synthesizers.
```

```
warnings.warn(DEPRECATION_MSG, FutureWarning)
```

```
c:\Users\TECH CLUB\AppData\Local\Programs\Python\Python313\Lib\site-
packages\sdv\single_table\base.py:134: UserWarning: We strongly recommend saving
the metadata using 'save_to_json' for replicability in future SDV versions.
```

```
warnings.warn(
```

```
Synthetic fraud generated: (5000, 7)
```

8. Combine Real + Synthetic Data

```
[61]: new_data=pd.concat([data,synthetic_fraud],ignore_index=True)
      print("After Combining, class Distribution:")
      print(new_data["IsFraud"].value_counts())
```

```
After Combining, class Distribution:
```

```
IsFraud
```

```
0    99000
```

```
1     6000
```

```
Name: count, dtype: int64
```

9. Re-encode & Retrain Model

```
[62]: new_data["TransactionDate"] = le1.fit_transform(new_data["TransactionDate"])

      new_data["TransactionType"] = le2.fit_transform(new_data["TransactionType"])
      new_data["Location"] = le3.fit_transform(new_data["Location"])
      X_new=new_data.drop("IsFraud",axis=1)
      Y_new=new_data["IsFraud"]
```

```

X_train,X_test,Y_train,Y_test=train_test_split(X_new,Y_new,test_size=0.
↳2,random_state=42)

model = RandomForestClassifier(random_state=42)
model.fit(X_train, Y_train)
y_test_pred= model.predict(X_test)
print("After adding Synthetic data once:")
print(confusion_matrix(Y_test,y_test_pred))
print(classification_report(Y_test,y_test_pred))

```

After adding Synthetic data once:

```

[[19809    0]
 [  234   957]]

```

		precision	recall	f1-score	support
	0	0.99	1.00	0.99	19809
	1	1.00	0.80	0.89	1191
	accuracy			0.99	21000
	macro avg	0.99	0.90	0.94	21000
	weighted avg	0.99	0.99	0.99	21000

10. Closed-Loop Iterative Training

```

[ ]: for i in range(2):
    print(f"--- Iteration {i+1} ---")

    # Generate synthetic fraud data
    synthetic_weak = synthesizer.sample(5000)

    # Encode synthetic data
    synthetic_weak["TransactionDate"] = le1.
    ↳fit_transform(synthetic_weak["TransactionDate"])
    synthetic_weak["TransactionType"] = le2.
    ↳fit_transform(synthetic_weak["TransactionType"])
    synthetic_weak["Location"] = le3.fit_transform(synthetic_weak["Location"])

    # Remove target column from features
    synthetic_weak_X = synthetic_weak.drop("IsFraud", axis=1)

    # Append to training data
    X_train = pd.concat([X_train, synthetic_weak_X], ignore_index=True)
    Y_train = pd.concat([Y_train, pd.Series([1]*len(synthetic_weak))],
    ↳ignore_index=True)

    # Safety check
    if "IsFraud" in X_train.columns:

```

```

X_train = X_train.drop("IsFraud", axis=1)

# Retrain model
model.fit(X_train, Y_train)

# Predict
y_test_pred = model.predict(X_test)

# Evaluate
print(confusion_matrix(Y_test, y_test_pred))
print(classification_report(Y_test, y_test_pred))

```

--- Iteration 1 ---

```
[[19809    0]
 [ 233   958]]
```

	precision	recall	f1-score	support
0	0.99	1.00	0.99	19809
1	1.00	0.80	0.89	1191
accuracy			0.99	21000
macro avg	0.99	0.90	0.94	21000
weighted avg	0.99	0.99	0.99	21000

--- Iteration 2 ---

```
[[19809    0]
 [ 232   959]]
```

	precision	recall	f1-score	support
0	0.99	1.00	0.99	19809
1	1.00	0.81	0.89	1191
accuracy			0.99	21000
macro avg	0.99	0.90	0.94	21000
weighted avg	0.99	0.99	0.99	21000